

AR Workbench Build Reference

Sep 23, 2026 · @Someone

Overview

The Denial & AR Workbench is a healthcare revenue-cycle-management (RCM) prototype for managing denied claims and open insurance/patient accounts receivable across multiple clinical clients. It models the full lifecycle of a claim from denial identification through assignment, follow-up, QA review, escalation, and financial resolution, for six operating roles: System Administrator, RCM Manager, Senior AR Analyst / Team Lead, AR Agent, QA Reviewer, and Client/Management Viewer.

It is a single-page vanilla JavaScript application — no build step, no framework — loaded as sequential scripts over a static JSON claims dataset (227 seeded claims across multiple clients, payers, and denial categories). All data mutations are simulated client-side via `localStorage`, making the app a fully interactive, self-contained demo: every workflow (assignment, QA approval, escalation, bulk actions, user management) is really executable, not mocked.

The build ships two ways: as a published Claude Artifact (live, versioned, shareable by link) and as a single self-contained downloadable HTML file (`LRN_Denial_AR_Workbench.html`) that runs offline with no server. Both are generated from the same source tree and kept in lockstep — every feature change is built, tested with Playwright, bundled, and republished to both simultaneously.

Roles, permissions, and access scoping

Every signed-in user has a role and, separately, an access grant that scopes which claims they can see. Roles gate which navigation items and actions exist at all; access grants narrow the data within those views.

Role	Label	Key permissions
admin	System Administrator	Everything — assign, edit claims, QA decisions, approvals, user management, audit, settings, all clients

Role	Label	Key permissions
manager	RCM Manager	Assign, edit claims, QA decisions, approvals, audit, all clients
lead	Senior AR Analyst / Team Lead	Assign, edit claims, QA decisions, all clients
agent	AR Agent	Edit claims only (their own assigned caseload)
qa	QA Reviewer	QA decisions, all clients
viewer	Client / Management Viewer	Read-only

A viewer's read-only access is further scoped to one of three levels, set by an admin in User Management:

1. **Client level** — sees every claim for one client across all its clinics and providers (e.g. Bo S → Summit Toxicology Labs).
2. **Clinic level** — sees only claims for one clinic (ordering practice) within a client (e.g. Renee K → Summit Toxicology Labs → Northgate Orthopedic Associates).
3. **Provider level** — sees only claims for one referring/rendering provider within a client (e.g. Marcus D → Highland Diagnostic Labs → Dr. Daniel Ngata).

When an admin adds a user, a cascading picker in User Management walks Access Level → Client → Clinic or Referring Provider, populated from the real distinct clinic and provider values found in the claims data (never free text). Scoping is enforced centrally: every view reads claims through a single `scopedClaims()` function that filters by allowed client(s), then narrows by clinic or provider when the signed-in user's access level calls for it, then by agent for the Agent role. Client, clinic, and provider-scoped users see a fixed badge naming their scope in place of the all-clients dropdown, so the boundary is always visible, not just enforced silently.

Custom users created through User Management can actually log in during the session (not just appear as a table row), so a newly granted access level is immediately testable end to end.

Data model and claim lifecycle

The app is driven by two JSON files. `claims.json` holds 227 claims, each carrying identification (claim ID, patient account, client, payer, financial class, panel type), providers (billing, rendering/referring, ordering practice/clinic), dates of service, financial fields (billed, allowed, insurance payment, patient payment, adjustments, insurance balance, patient balance, total balance), denial detail (code, category, reason, date, root cause), workflow state (status, category, assigned agent, priority, follow-up dates, aging), a per-claim `activity[]` audit log, `followUps[]`, QA state (`qa{}`), agent-initiated `agentRequests[]` (escalations/reassignments), CIP case records, and recovery/AR-remediation fields (expected vs. actual payment, variance, underpayment, appeal flag, recovered amount, remaining AR). `meta.json` holds all admin-editable reference data — clients, payers, panel types, agents, QA reviewers, denial categories, workflow statuses, aging buckets, financial classes, timely-filing-limit thresholds, workflow templates, demo user personas, the comments framework, revenue-confidence tiers, AR queue definitions, and the non-collectible denial code list.

Financial closure is derived, not stored as a single flag: a claim is financially closed when both `remainingAR` and `patientBalance` are at or below zero (`isFinanciallyClosed`); it is work-complete when it's financially closed or its workflow status is `Completed` (`isWorkComplete`); it still has open insurance AR whenever `insuranceBalance` is greater than zero (`isOpenInsuranceArClaim`) — a distinction that matters because a claim can be marked `Completed` by QA while still carrying an outstanding insurance balance the payer hasn't paid out yet.

Staleness is measured by `daysSinceLastTouch`, which looks at the most recent entry in a claim's `activity[]` log rather than its `lastFollowUp` field (which is always null for a claim that has never been assigned, so it can't distinguish a fresh denial from a long-neglected one). Every claim has at least one activity entry from intake, so this works uniformly across touched and never-touched claims.

AR queue classification (`classifyQueue`) sorts every claim into one of roughly eleven top-level AR queues (with sub-queues) based on remaining balance, patient balance, recovered/adjusted amounts, and denial category, and separately flags claims as non-collectible when their denial code appears on the admin-editable Non-Collectible Denial Codes list, and as due for re-follow-up once 45+ days have passed since `lastFollowUp` or `nextFollowUp` has lapsed — a rule that only applies once a claim has been worked at least once, distinct from the untouched-claim staleness rule above.

Navigation map

View	Roles	What it does
Dashboard	All	KPI overview — AR totals, denial trends, aging, recovery metrics
Clients	Admin	Client roster and configuration
Data Processing	Admin, Manager	Claims intake / ETL simulation
Work Queue	Admin, Manager, Lead, Agent	Master claim list with full filtering, sorting, export
Assignment Management	Admin, Manager, Lead	Unassigned Claims (45+ day-untouched, open-balance), Create Assignment Batch, and Assigned Claims — Bulk Reassign
My Work	Agent, Lead	An agent's personal assigned caseload
Follow-Up Management	Admin, Manager, Lead, Agent	Claims due or overdue for re-follow-up
QA Verification	Admin, Manager, Lead, QA	QA review queue with bulk filter/select/approve
CIP & Escalations	Admin, Manager, Lead	Corrective Implementation Plan case tracking
Escalation & Reassignment Requests	Admin, Manager, Lead	Agent-initiated requests, with bulk filter/select/resolve
Client CIP	Viewer	Read-only CIP visibility for scoped clients
Analytics	All	Charts — recovery by payer/panel/agent, denial trends, aging
Reports	All	Summary reports, including Panel Type Summary
Users	Admin	User Management — create/edit users and access grants

View	Roles	What it does
Audit	Admin, Manager	Cross-claim activity/audit trail
Settings	Admin	Master File Maintenance — admin-editable reference lists

Every route is declared once in `App.NAV` with its allowed roles; the sidebar, routing, and permission checks all read from that single source, so a role's visible navigation and its enforced access are guaranteed to match.

Work Queue, Assignment Management, and Follow-Up logic

Work Queue is the master claim list — every claim within a user's access scope, with filtering by client, payer, panel type, denial category, agent, status, aging bucket, and more, full sorting, pagination, and CSV export.

Assignment Management has three parts that intentionally draw from different populations of the same underlying data:

- **Unassigned Claims** shows claims with `workflowStatus = 'Unassigned'`, an open insurance balance, and untouched for 45+ days (via `daysSinceLastTouch`). This is the neglect-surfacing view — it deliberately excludes fresh denials that haven't sat long enough to be a problem yet, and carries a "Days Untouched" column.
- **Create Assignment Batch** draws from the full unassigned-and-open pool (`unassignedOpen`), with no 45-day restriction, since proactively batch-assigning claims the day they're identified is normal, desired workflow — it shouldn't be gated by a staleness threshold meant for spotting neglect.
- **Assigned Claims — Bulk Reassign** shows every claim with an assigned agent and an outstanding insurance balance, including claims marked `Completed` whose payer hasn't paid out yet (closed, zero-balance claims are excluded). This is broader than the Agent Workload chart's own "open claims" count, which still excludes work-complete claims — an intentional difference between "what's reassignable" and "what's actively open work."

Follow-Up Management surfaces claims due or overdue for re-follow-up, using the `arqRefollowupDue` rule (45+ days since `lastFollowUp`, or `nextFollowUp` already passed) — a separate rule from the Unassigned Claims staleness check above, since it applies only to claims that have already been worked at least once.

QA Verification and Escalation & Reassignment Requests

QA Verification reviews claims awaiting a QA decision. An agent can never approve their own work (checked against `assignedAgent`); approving sets the claim to `Completed` and marks any linked escalation as approved, while rejecting routes it back as `QA Rejected`. The queue supports full filtering (client, payer, panel, category, agent, reviewer, escalation flag, status) plus bulk selection: rows that can't be bulk-approved (a reviewer's own work, or a claim not actually awaiting decision) render with no checkbox and are excluded from "select all," and an "Approve Selected" action confirms the batch (showing count and total insurance AR) before applying each decision silently and reporting one summary toast.

Escalation & Reassignment Requests flattens every claim's agent-initiated requests (escalations and reassignment asks) into one queue, filterable by client, payer, request type, agent, requester, AR queue, and status. It mirrors the QA queue's bulk pattern: already-resolved requests can't be re-selected, and a "Resolve Selected" action opens a modal summarizing the escalation/reassignment breakdown, requires a shared resolution note, and resolves the whole batch in one step.

CIP & Escalations tracks formal Corrective Implementation Plan cases arising from approved escalations, visible to Admin, Manager, and Lead; a read-only **Client CIP** view gives scoped Viewers visibility into CIP status for their own client, clinic, or provider.

Analytics, Reports, and Master File Maintenance

Analytics charts recovery and denial performance across dimensions — by payer, panel type, agent, denial category, and aging — including a dedicated Agent Workload view and a Recovery by Panel Type chart, all respecting the signed-in user's access scope.

Reports surfaces summarized, exportable views of the same data, including a Panel Type Summary alongside client, payer, and aging summaries.

Master File Maintenance (in System Settings, Admin only) is where the admin-editable reference lists that drive the rest of the app live: Non-Collectible Denial Codes, Denial Categories, Panel Types, Denial Root Cause Options, and Fix/Resolution Options, each managed as an add/remove chip list with CSV download and upload. It also includes a two-column Denial Code → Denial Category mapping table and a Fix/Resolution-by-Claim-Status configuration. Every one of these lists is backed by `localStorage` overrides layered on top of the seeded defaults in `meta.json`, so admin edits persist for the session without touching the underlying dataset.

Technical architecture

The app is a single global namespace, `window.App`, with `App.views.*` (one render function per route), `App.actions.*` (state-mutating operations), and shared helpers, loaded via ordered, non-module `<script>` tags with no build step:

```
data.js → core.js → actions.js → charts.js → dashboard.js → workqueue.js →
claimdetail.js → assignment.js → mywork.js → qaapproval.js → analytics.js →
admin.js → client.js → main.js
```

A handful of shared components carry nearly all of the UI: `App.createTable` (sortable, paginated, exportable, selectable tables, with an optional `rowSelectable` predicate for conditional bulk-eligibility), `App.multiSelectField` / `wireMultiSelect` (the standard filter-dropdown popover used on every filter bar), `App.openModal` / `closeModal`, and `App.confirmAction` (the confirm-before-destructive-action dialog used ahead of every bulk or irreversible operation).

Persistence is simulated entirely client-side: `App.saveOverride(claimId, patchFn)` mutates a claim in memory and writes the whole updated claim object to a `localStorage` key, merged back over the seed data on load; `App.loadLocal` / `saveLocal` provide the same pattern for admin-editable master lists and session-created users. There is no backend — this is a demo/prototype data layer, not a production persistence model.

Testing and delivery: every feature is verified with Playwright browser automation against both the raw source (via a local static server) and the built bundle (via `file://`), covering the new feature end to end plus a regression pass on anything it touches. `build_bundle.py` inlines every script, stylesheet, and JSON data file into the single self-contained `LRN_Denial_AR_Workbench.html`. Each change is published to the live Claude Artifact by diffing the about-to-change files against the last known-good published version (to catch drift) before publishing the update — so the Artifact and the downloadable bundle are always kept in sync, both tested, both versioned.

Claim Workspace, Escalations & Remaining Modules

The Claim Detail workspace

Every queue in the app (Work Queue, My Work, Assignment, QA, CIP Escalations, Audit, Agent Requests) opens into the same single-claim screen. Its header carries the claim ID, current workflow status, AR queue classification, priority, TFL-at-risk flag, any pending escalation/reassignment request, and the key identifiers (client, payer, panel, date of service, patient account, rendering provider, ordering practice). Below that, a four-stage workflow stepper (Identified → Assigned → QA Verification → Completed) shows where the claim sits, alongside the full named stage path for its denial-category workflow template (for example, a coding-related denial's path runs Identified → Assigned → Coding Review → Correction Required → Corrected Claim Submitted → Payer Follow-Up → Payment/Resolution → QA → Closed).

The screen is organized into tabs, shown only when relevant: Overview (financial summary), CPT/Line Detail, Denial Info, Follow-Ups (with a count), Activity Timeline (the full audit trail for that claim), QA Review (once the claim has a QA record), and CIP Case(s) (once an escalation has been opened on it, labeled with a count if there's more than one).

Role-gated action buttons sit in the header: Assign/Reassign for anyone with assignment rights, and — for an AR Agent on an open, non-QA-pending claim — Log Follow-Up Note, Escalate to Supervisor, and Request Reassignment. The last two are how an agent asks a Team Lead, RCM Manager, or Administrator for help; those roles already have Assign/Reassign and respond from the Escalation & Reassignment Requests queue, so the request-side buttons only appear for the agent role.

Logging a Follow-Up Note

This is the single entry point for an AR agent to record work done on a claim — the actual fix happens in the practice management system or with the payer; this just logs it. The modal captures the client's Comments Framework fields: Claim Type (Primary/Secondary), Follow Up Type (Review/Online/Call), Claim Status (Paid, Denied, Not Received, In Process, Paid to Patient, Patient Responsibility, On Hold), a Denial Root

Cause (only asked when Claim Status is Denied), a Fix/Resolution chosen from a list scoped to the selected Claim Status, a free-text Follow Up Comment, and a Next Follow-Up Date (defaulted to 7 days out, driving the Re-follow-up Required queue).

Saving the note automatically submits the claim to the QA Verification Queue — there is no separate "Submit for QA" step. Choosing "CIP - Client Escalations" as the Fix/Resolution reveals a second block of fields (CIP Category, Required Information, and a CIP Comment aimed at the client) and, on save, opens a CIP case in "Pending Approval" status alongside the ordinary QA submission — the follow-up note itself is a historical record of the request, while the CIP case is the live, actioned copy that then moves through its own approval lifecycle (see the next section).

CIP — Client Escalations: the full lifecycle

A CIP (Corrective Implementation Plan) case is how the team asks a client for information it can't get any other way, with a formal audit trail on both sides. It moves through five states:

1. **Pending Approval** — an AR agent has just logged a follow-up with Fix/Resolution "CIP - Client Escalations," opening the case. It is not yet visible to the client.
2. **Sent to Client** — a Team Lead, RCM Manager, or Administrator reviewed the request from the CIP Escalations queue and approved it; the CIP Category, Required Information, and CIP Comment become visible to the client on their Escalation Requests page.
3. **Client Responded** — the client submitted the requested information (text plus up to 3 attachments), routing the case back to the CIP Escalations queue for review.
4. Back to **Sent to Client** (round + 1) — if the reviewer marks the response incorrect or insufficient, with a reason recorded for the client to see.
5. **Returned to Agent** — once a response is accepted (or a request is rejected before ever reaching the client), the claim reopens to "Assigned" status so the agent can pick up where the case left off.

The CIP Escalations queue (Team Lead / RCM Manager / Administrator) shows a live count of cases at each stage and supports the same bulk pattern as QA Verification and Escalation & Reassignment Requests: rows can be multi-selected and resolved via "Bulk Approve Selected" or "Bulk Reject / Send Back Selected," with a shared note applied across every case touched and each row getting the action appropriate to its own stage (a mixed selection of Pending Approval and Client Responded cases is handled correctly in one action). On the client side, the Escalation Requests page lists everything awaiting

their response, submitted and under review, or resolved, and offers two ways to respond: individually with a text reply and file attachments, or in bulk via a downloadable CSV response template that can be filled in Excel and uploaded back to update every open item at once.

An internal **Escalate to Supervisor** (available to an AR agent from the Claim Detail screen) is a distinct, simpler mechanism: it routes the claim to QA for supervisor review and appears in the Escalation & Reassignment Requests queue, but it never opens a CIP case and never involves the client — use it for "I need a second set of eyes," and use a CIP escalation for "I need information only the client has."

Automatic Adjustment for non-collectible denials

Not every denial needs an agent's attention. When a claim's denial code matches the admin-editable Non-Collectible Denial Codes master list, a Manager, Lead, or Administrator can trigger Automatic Adjustment on it directly from the Work Queue (individually or in a batch) — the remaining insurance balance is written off in one step, with no agent ever touching the claim. The write-off is recorded on the claim's activity log as a "System / Automation" entry, distinct from every other activity entry (which is always attributed to a signed-in person), so the audit trail is clear about what was a human decision versus an automated one. This is the "no-touch" counterpart to the ordinary assign-and-work path: it exists specifically to keep genuinely uncollectible denials from consuming agent capacity.

Data Processing, Client Management, and Audit Logs

Data Processing (Admin, Manager) simulates the weekly ETL refresh: it shows the last refresh date, source period, and record count, and a "Run Data Processing" button walks through a simulated processing sequence (reconciling source data, then running the Denial & Payment Analysis) before confirming completion. Below that, the Denial Analysis Report surfaces the current cycle's most common denial codes grouped and ranked as actionable insights, each linking through to the matching claims.

Client Management (Admin) lists every client with its eligible claim count, outstanding AR, and recovered total, and lets an admin deactivate or reactivate a client. This is a lightweight session-only toggle (an inactive badge on the client card) rather than a hard delete — claims for a deactivated client remain in the data, so this models client offboarding/onboarding status rather than actually removing their data.

Audit Logs (Admin, Manager) is the flattened activity trail across every claim in scope — every entry any action in the app has ever written (assignments, follow-ups, QA decisions, escalations, CIP case events, reassignments, automatic adjustments) in one searchable, filterable table (by user, action, and client), each row linking back to its claim. This is the same per-claim activity log shown on the Claim Detail screen's Activity Timeline tab, just aggregated across the whole book of business.

Cross-cutting utility features

A handful of conveniences are available on every major table in the app (Work Queue, My Work, Follow-Up, Assignment, QA, CIP Escalations, Agent Requests, Audit), not just the queue-specific behavior described elsewhere:

- **Saved views** — a filter combination can be saved by name and reselected later, remembered per browser for that queue.
- **CSV export** — every table can export its current (filtered, sorted) rows to a CSV file, matching exactly what's on screen rather than the full unfiltered dataset.
- **Column visibility** — columns can be shown or hidden per table, so a user can trim a wide table down to what they actually look at.
- **Assignment Batches** — in Assignment Management, a set of filter criteria plus an agent and due date can be saved as a named batch, which then shows up in a running Assignment Batches log for that session (claim count, total insurance AR, status, completion) rather than disappearing once applied.

These are consistent by design: they're built once as shared components (see the Developer Documentation's Architecture tab) and reused everywhere, so the experience of exporting, filtering, or customizing a table is the same regardless of which queue you're in.